DMMST Version 2 — explained from zero

Parsa

This document explains, without assuming any background, what survival analysis is, what our model does, what every loss and dataset is, what we ran in version 2, and what came out. Numbers are from notebooks 01–06 (the language-model notebook 07 is not included).

Contents

1 The problem **3**
- 1.1 What is survival analysis? . . . 3
- 1.2 What does a survival model output? . . . 3
- 1.3 One event or several events? . . . 3

2 Our model **3**
- 2.1 The idea in one paragraph . . . 3
- 2.2 The numeric embedding (paper §2.2) . . . 4
- 2.3 Representation: how to get one patient vector . . . 4
- 2.4 How the survival head works (L_{PCH}) . . . 4

3 The losses **4**
- 3.1 The recipes we tested . . . 5

4 The data **5**

5 How we measured (the protocol) **6**
- 5.1 Splits . . . 6
- 5.2 Tuning (notebook 01) . . . 6
- 5.3 Early stopping and "patience" . . . 6
- 5.4 The scores . . . 6

6 The models we compared against **7**

7 What each notebook did **7**

8 The results, in plain words **7**
- 8.1 Are we better than the other models? (02, 03) . . . 7
- 8.2 Do the extra losses help? (05) . . . 8
- 8.3 Does the numeric embedding help? (04) . . . 8
- 8.4 Does the regression head work? (06) . . . 8
- 8.5 What this means for the paper . . . 9

9 Problems we found and fixed along the way **9**

10 The code, briefly **9**
10.1 How one run works 9
10.2 Configuration (`conf/`, Hydra) 10
10.3 The library (`sat/`) 10
10.4 Running many experiments (`scripts/`) 11
10.5 The notebooks (`notebooks/version_2/`) 11
10.6 Tests (`tests/`) 11

11 Where everything is **11**

1 The problem

1.1 What is survival analysis?

We want to predict **when** something will happen to a patient — death, relapse of a cancer, recovery after a transplant. That "something" is called an **event**, and the time until it happens is the **event time**.

It is not a normal prediction problem because of **censoring**: for many patients we never see the event. A study runs for, say, 10 years; a patient who is still alive at the end, or who moved away after 3 years, has no event time. We only know "the event had not happened by time T". Such a patient is **censored** at time T. Throwing censored patients away would bias everything (they are often the healthiest ones), so survival models are built to use the partial information they carry.

1.2 What does a survival model output?

Instead of one number, a survival model usually outputs a **survival curve** for each patient:

$$S(t) = \text{the probability that the event has } \textit{not} \text{ happened by time } t.$$

S starts at 1 (at time 0 nothing has happened) and goes down. A high-risk patient's curve drops fast, a low-risk patient's curve drops slowly. From the curve one can read a risk at any time, or an expected event time.

1.3 One event or several events?

- **Single event**: one kind of event, e.g. death.
- **Multiple events**: several kinds per patient, e.g. after a bone-marrow transplant: recovery, an adverse event, relapse, death. They can be
  - **competing**: only the first one is seen (if you die of A you cannot later die of B);
  - **semi-competing / non-exclusive**: several can happen to the same patient, in some order (recovery, then relapse, then death).

Our paper (DMMST) is about a model that handles **multiple events**.

2 Our model

2.1 The idea in one paragraph

Each patient is a list of features (age, lab values, treatment, ...). We turn every feature into a small vector (a **token**), feed all tokens of a patient into a **transformer** (the same kind of network used in language models), and get one vector that summarises the patient. On top of that vector sit **heads** — small networks that produce the outputs:

- the **survival head** outputs one survival curve per event;
- the **regression head** (optional) outputs one predicted event time per event.

2.2 The numeric embedding (paper §2.2)

How do you give a number like "age = 63" to a transformer? Two options:

- **Discretise** it: cut age into bins (e.g. 60–65) and treat the bin as a word. Simple, but loses precision.
- **Numeric embedding** (what the paper proposes): every feature has its own learned vector, and that vector is **multiplied by the (standardised) value**. Age 63 and age 64 then give almost the same input, as they should.

We also tried **quantile** scaling (the value is replaced by its rank among all patients before the multiplication).

2.3 Representation: how to get one patient vector

The transformer outputs a vector per token per layer. We tried different ways to combine them: over **layers** (sum, mean, concatenate, or a special [CLS] token), over **tokens** (max or mean), and using **all layers** or only the **last** one.

2.4 How the survival head works (L_{PCH})

Time is cut into a few intervals (e.g. 0–14 days, 14–18 days, ...). In each interval the head predicts a **hazard**: how likely the event is in that interval if it has not happened yet. Multiplying the "no event" chances interval by interval gives the survival curve. This is called a **piecewise-constant hazard (PCH)** model.

The model is trained with L_{PCH}, the likelihood of what we actually observed:

- for a patient with an event at time T: "survive until T, then the event happens at T";
- for a patient censored at time T: "survive until T" (nothing more is claimed).

This is how censored patients are used correctly.

3 The losses

A **loss** is the number the model tries to make small during training. L_{PCH} alone already gives a working model. The paper adds more losses; testing whether they help is a large part of version 2.

Loss	Paper	What it asks of the model, in plain words
L_{PCH}	Eq. 3	Make the observed event times (and censoring times) likely. The base loss.
L_{rank}	Eq. 4	**Between patients**: if patient A had the event before patient B, A should have been predicted as riskier than B (for the same event).
L_{mul}	Eq. 5	**Within one patient**: if a patient had event 1 before event 2, event 1 should have been predicted as riskier at that time. Needs ≥ 2 events.
L_{MAE}	Eq. 6	For the regression head: the predicted time should be close to the real time.
L_{MM}	Eq. 7	For the regression head: the predicted times of the different events should agree with each other ("mismatch" penalty). Needs ≥ 2 events.

Best guess (Eq. 6). For a censored patient we do not know the true time, only that it is later than T. The "best guess" is an estimate of the real time made from all training patients (Kaplan–Meier curve): "patients who were still event-free at T had their event on average at time X". L_{MAE} can either ignore censored patients (R1) or use their best guess (R2).

Weight and σ. L_{rank} and L_{mul} have a strength (weight) and a sharpness (σ). These were tuned. The paper writes Eqs. 4–5 with the wrong sign; we use the correct direction (as in DeepHit) and a test checks it.

3.1 The recipes we tested

Survival head		Regression head	
S1	L_{PCH} only	R1	L_{MAE}, observed events only
S2	$L_{PCH} + L_{rank}$	R2	L_{MAE} with the best guess
S3	$L_{PCH} + L_{mul}$ (multi-event)	R3	R2 + L_{MM}, observed time as reference
S4	$L_{PCH} + L_{rank} + L_{mul}$ (multi-event)	R4	R2 + L_{MM}, best guess as reference

"—" in a table means the recipe needs ≥ 2 events and does not exist on that dataset.

We also removed one loss that was in earlier versions, **MMV** (from another paper, UniSurv): it is not in our paper and does the same job as our regression head.

4 The data

Dataset	Real?	Patients	Events	What it is
METABRIC	real	1,904	1: death	breast-cancer patients; 9 features (genes + clinical)
SUPPORT	real	8,873	1: death	seriously ill hospital patients; 14 features
EBMT	real	2,279	4: recovery, adverse event, relapse, death	bone-marrow transplant patients; only 4 categorical features
hsa synthetic	simulated	5,000	2, both can happen	from Tjandra et al. (2021); about 70% censored at the end of the study
DeepHit synthetic	simulated	30,000	2, competing	from the DeepHit paper (Lee et al. 2018)

METABRIC and SUPPORT are the standard datasets of SurvTRACE, our closest competitor, so we can compare with its published numbers. **SEER** was planned as a large real dataset, but because of the updated SEER+ data-access policies we could not get access to it.

5 How we measured (the protocol)

5.1 Splits

Each dataset is split at random into **60% training** (the model learns from it), **10% validation** (used to choose settings and when to stop training) and **30% test** (used only for the final numbers). This is done **10 times** with different random splits, and every model gets the same 10 splits. Results are written as **mean (standard deviation)** over the 10 splits — e.g. 0.667 (0.013). The standard deviation shows how much the result moves from split to split.

5.2 Tuning (notebook 01)

Every model has settings (learning rate, network size, ...). We tried many combinations on the **validation** split and kept the best — for every model, not only ours, so the comparison is fair. Then, with the chosen network, we tuned the loss weights (L_{rank}, L_{mul}, L_{MM}).

5.3 Early stopping and "patience"

During training the model is checked on validation regularly. If it has not improved for a number of checks (the **patience**), training stops. We use 30 for our loss and regression experiments (Section 9).

5.4 The scores

Score	Meaning	Good value
C_{td}	Of all pairs of patients, how often is the one who had the event first predicted as riskier? Measured at the 25%, 50% and 75% points of the event times, then averaged.	higher; 0.5 = coin flip
Brier score	Squared error of the predicted probability "event by time t" versus what happened.	lower
IBS	Brier score averaged over the whole follow-up.	lower
IBLL	Like IBS, but with log-loss.	lower
AUC	Another standard way to measure ranking at a time point.	higher
Antolini C	A C_{td} version that uses the whole curve.	higher
D-calibration p	Tests whether predicted probabilities match observed frequencies. Large test sets make it very strict.	higher (> 0.05 = no problem found)
MAE	Average error of a predicted event time, in the dataset's time unit. Censored patients are handled with the best guess ("margin") or "pseudo-observations".	lower
Harrell C	Like C_{td} but for a predicted *time*: are earlier events predicted earlier?	higher

In short: C_{td} = **does the model put patients in the right order; IBS = are its probabilities right.**

6 The models we compared against

Model	What it is
SurvTRACE	A transformer survival model (2022), run with the authors' own code. Our closest competitor.
DeepHit	Neural network that predicts the event-time distribution directly (2018).
DSM	Deep Survival Machines: a mixture of simple distributions (2021).
MENSA	A recent multi-event model (2024).
RSF	Random Survival Forest: many decision trees. A strong classic.
PC-Hazard	A simple neural network with the same piecewise hazard as our head.
DeepSurv	A neural version of the Cox model.
Cox	The classic statistical survival model (linear).

DeepHit, DSM and MENSA use our transformer with their own head; the others are separate models. On multi-event data, single-event models are trained once per event.

7 What each notebook did

Notebook	What it did	Runs
01 Tuning	chose the settings of every model and the loss weights, on validation only	576
02 Tabular benchmark	all 9 models on METABRIC and SUPPORT (single event)	180
03 Multi-event benchmark	all 9 models on EBMT, hsa synthetic, DeepHit synthetic	270
04 Encoding / representation	numeric embedding vs quantile vs bins; 13 ways to pool the transformer	320
05 Survival losses	S1–S4 on all datasets	160
06 Regression head	R1–R4 alone, and every S × R together with the survival head ("joint")	720

8 The results, in plain words

8.1 Are we better than the other models? (02, 03)

C_{td}	METABRIC	SUPPORT	EBMT	hsa synth.	DeepHit synth.
Best other model	SurvTRACE 0.678	SurvTRACE 0.638	RSF 0.549	SurvTRACE 0.868	DeepSurv 0.762
Our best recipe	0.672	0.620	0.537	**0.868**	0.744
Our rank (of 9, S1)	5	4	6	3	5

Plainly: our model is good but not the best. It is always in the better half and ties SurvTRACE on hsa synthetic, but on the other four datasets something else is slightly better (by 0.006–0.018). Our **calibration** (IBS) is good — close to the best on four datasets, and better than SurvTRACE on METABRIC. A sanity check also passed: our run of SurvTRACE gets almost its published numbers (0.678 vs 0.691, 0.638 vs 0.640), so the setup is right.

EBMT is hard for every model (all between 0.50 and 0.55): its 4 features say little about the outcome.

8.2 Do the extra losses help? (05)

C_{td}	METABRIC	SUPPORT	EBMT	hsa synth.	DeepHit synth.
S1: L_{PCH}	0.667	0.615	0.529	0.849	0.743
S2: $+L_{rank}$	0.670	0.620	0.535	0.868	0.743
S3: $+L_{mul}$	—	—	0.530	0.859	0.742
S4: +both	—	—	0.537	0.867	0.742

- L_{rank} **helps a little, on every dataset.** The gains are small; statistically clear only on hsa synthetic.
- **Their real benefit is reliability.** With L_{PCH} alone, training sometimes goes wrong on one split: on hsa one split reached only 0.71 (the others about 0.86), and on EBMT several splits had very bad probabilities (Brier up to 0.6). With $L_{rank} + L_{mul}$ (S4) this did not happen: EBMT's IBS went from 0.338 to 0.233 and became much more stable.
- On DeepHit synthetic nothing changes — L_{PCH} alone is enough there.

8.3 Does the numeric embedding help? (04)

- METABRIC: yes — 0.666 versus 0.642 with bins.
- SUPPORT: no — bins are slightly better (0.627 versus 0.617).
- How the transformer outputs are combined hardly matters (all within ±0.01), except that the [CLS] token is clearly worst.

8.4 Does the regression head work? (06)

- **R2 (best guess) is better than R1 (observed only)**: it orders the predicted times better on every dataset (Harrell C, e.g. 0.57 → 0.66 on hsa) and has a smaller error on 4 of 5 datasets.
- L_{MM} **(R3/R4)** changes little.
- **Training both heads together** ("joint"):
  - does not hurt the survival predictions, as long as L_{rank} is used (on hsa: 0.84–0.85 with L_{rank}, 0.72–0.82 without);
  - on hsa it even makes the predicted times better (Harrell C 0.69 on average, up to 0.79, versus 0.66 alone);
  - on DeepHit synthetic the predicted times get worse (0.55 versus 0.70), because joint models pick their best checkpoint by the survival score, not by the time prediction.

8.5 What this means for the paper

Claim	Supported?
"Better accuracy than other models"	**No** — competitive, upper half
"Well calibrated"	Mostly (not on EBMT)
"L_{rank} and L_{mul} improve the model"	Yes, but as **stability and calibration**; accuracy gains are small
"The numeric embedding is better than bins"	On METABRIC yes, on SUPPORT no
"The regression head works; the best-guess loss helps"	Yes
"Training both heads together helps"	On hsa yes; neutral elsewhere; worse time prediction on DeepHit synthetic

9 Problems we found and fixed along the way

Version 2 was run on Kaggle; several issues were found from the results and fixed before the final numbers. They are listed so you know the final results are trustworthy.

Problem	What happened	Fix
SurvTRACE crashed	its 2021 code used a NumPy name removed in NumPy 2	restored the name
Ranking loss gave NaN	on whole-number times a formula ran outside its interval and overflowed	kept it inside the interval
Qwen (LLM) crashed	a library version clash on Kaggle	bypassed the check
IBS/IBLL/AUC missing	some test patients were followed longer than any training patient	handle them correctly (cap the time)
Brier wrong on hsa	70% of hsa patients (censored at the end) were silently dropped from the Brier score of the neural models	keep them
Joint models collapsed	the regression loss measured in days (hundreds) drowned the survival loss	measure time as a fraction of follow-up
Some joint runs never learned	they start slowly and were stopped too early (patience 10)	patience 30 (also for notebook 05, to be fair)
"Last layer" runs crashed	wrong layer index (notebook 04)	use the real last-layer number
Kaggle killed long sessions	runs went past the 12-hour limit and nothing was saved	notebooks stop at 11 h 20 and save

10 The code, briefly

The code lives in the `dmmst` repository (a fork of the open-source **SAT** library, extended by us). You never need to call most of it directly: the notebooks do. Here is what each part is for.

10.1 How one run works

Every experiment is a short pipeline of Python programs, each started with a **configuration**:

`prepare_data` → `train_tokenizer` → `train_labeltransform` → `finetune`

- `prepare_data` reads the data; `train_tokenizer` turns features into tokens;
- `train_labeltransform` makes the time intervals and, per split, the best guesses;
- `finetune` trains the model, scores it and saves `metrics.json`.

The first three run once per dataset (the label step once per split); `finetune` runs once per model, recipe and split. Baselines use `baselines.py` or `survtrace_official.py` instead of `finetune`.

10.2 Configuration (`conf/`, Hydra)

Settings are not written in the code but in YAML files, combined by the **Hydra** library. A run is described by a few overrides on the command line, e.g.

```
python -m sat.finetune experiments=multievent/survival dataset=hsa_synthetic_me
            tasks=v2_survival v2_rank_coeff=1.0 seed=3 split_seed=3
```

Folder / file	What it holds
`conf/experiments/`	one group per dataset family; `defaults.yaml` holds all shared settings (`v2_rank_coeff`, `v2_mul_coeff`, `v2_l1_type`, ...)
`conf/tasks/v2_*.yaml`	which heads the model has (survival, regression, or both)
`conf/tasks/losses/v2.yaml`	the loss recipe: which losses are on and their weights
`conf/tasks/metrics/v2.yaml`	the metrics computed at evaluation

10.3 The library (`sat/`)

Path	What it does
`prepare_data.py`, `data/`	read each dataset into one common format; `data/splitter.py` makes the exact 60/10/30 splits from a seed
`train_labeltransform.py`	time intervals and, **from the training split only**, the Kaplan–Meier best guesses and censoring weights
`models/heads/embeddings.py`	the numeric embedding (feature vector × value)
`models/heads/mtl.py`	the whole model: transformer + the configured heads
`models/heads/survival.py`, `regression.py`	survival head (hazards → curve) and regression head (predicted time)
`loss/survival/nllpchazard.py`	L_{PCH}
`loss/ranking/sample.py`, `multievent.py`	L_{rank} (between patients) and L_{mul} (between events of one patient)
`loss/regression/l1.py`, `mismatch.py`	L_{MAE} (R1/R2) and L_{MM} (R3/R4)
`loss/factory.py`, `meta.py`	build the recipe: only losses with weight > 0, added up
`finetune.py`	trains (Hugging Face Trainer), early-stops on validation C_{td}, scores validation and test, writes `metrics.json`
`evaluate/survtrace_metrics.py`	C_{td} and Brier at the 25/50/75% time points
`evaluate/v2_metrics.py`	IBS, IBLL, AUC, Antolini, calibration, MAE, Harrell C
`evaluate/shared.py`	the same scoring for the baselines
`baselines.py`	Cox, RSF, DeepSurv, PC-Hazard (scikit-survival / pycox)
`survtrace_official.py`	the authors' SurvTRACE code on our splits
`llm.py`	the language-model experiments (notebook 07)
`utils/km.py`	Kaplan–Meier "best guess"

10.4 Running many experiments (scripts/)

File	What it does
runner.py	the **Runner**: launches runs in parallel, skips finished ones, retries failures once, stops safely before Kaggle's time limit, collects all results into one table, paired significance tests
v2.py	the **version-2 plan**: datasets, how each model is launched, search grids, loss recipes, reading tuned.json, tables
verify_protocol.py	checks the protocol (split sizes, no test data in training files)
prepare_ebmt.py, prepare_public_synthetic.py, fetch_survtrace.py	download or build EBMT, the synthetic datasets and the SurvTRACE code

10.5 The notebooks (notebooks/version_2/)

The notebooks are **generated** by build_notebooks.py from shared cells, so they all set up the same way. On Kaggle each notebook (1) clones the code from GitHub, (2) installs packages, (3) copies back finished runs from an attached previous output, (4) reads tuned.json, (5) builds its list of runs and gives it to the Runner, (6) writes tables and a zip. Each run leaves two files: <run>.json (all metrics for validation and test) and <run>.meta.json (exactly how it was run, including the code version).

10.6 Tests (tests/)

159 automated tests check, among other things, that our metrics equal the reference libraries, that the ranking losses push in the right direction, and the edge cases fixed during version 2. They run with pytest.

11 Where everything is

- Results of each notebook: results/version_2/<notebook>/
- Figures and reports: results/version_2/analysis/
  - REPORT.md — short comparison of our losses versus other models;
  - report_latex/report.pdf — 3-page report;
  - report_latex/full_report.pdf — 16-page report with every table;
  - report_latex/explained.pdf and EXPLAINED.md — this document.
- The plan of version 2: notebooks/version_2/VERSION.md
- Not yet included: notebook 07 (the language-model part of the paper, §3) and sequential data (§4.2, planned for version 3).